Programming Assignment 3: Implementing Shor's Factoring Algorithm

Spring 2026 — Assoc. Prof. Dr. Mehmet Cengiz Onbaşlı

Assignment due: 01/05/2026, 23:59

Overview

This assignment asks you to implement Shor's factoring algorithm *end-to-end* in Qiskit, from the construction of modular exponentiation unitaries through to classical post-processing and factor extraction. Unlike the weekly coding exercises, *no step of the quantum computation is bypassed*: you will build the actual modular exponentiation gate, wire it into a phase estimation circuit, extract the period via continued fractions, and apply the classical factor extraction step.

A complete Shor's algorithm implementation in four connected parts:

1. **Part 1.** The modular exponentiation unitary $U_a |y\rangle = |a \cdot y \bmod N\rangle$ as a Qiskit `UnitaryGate`.
2. **Part 2.** The full Quantum Phase Estimation (QPE) circuit using controlled-$U_a^{2^k}$ gates and inverse QFT.
3. **Part 3.** Classical post-processing: continued fractions, period validation, and GCD-based factor extraction.
4. **Part 4.** Generalisation: factor $N = 21$ with parameter choices you justify, and a success-rate sweep over all valid bases.

Solutions that bypass the modular exponentiation gate (e.g. by preparing the periodic state directly with Hadamard gates, as shown in the Week 9 lecture notebook) will receive **zero credit** for Parts 1 and 2, even if the downstream pipeline produces correct factors.

Deliverables

- **Completed Jupyter Notebook** `PA3_Shors_Algorithm_⟨StudentID⟩.ipynb` with all code cells filled in, all auto-check assertions passing, and all explanation markdown cells completed.
  Example filename: `PA3_Shors_Algorithm_0064384.ipynb`
- **Exported PDF** of the completed notebook showing all code and output. Use *File → Export Notebook As → PDF* or print-to-PDF.

- Submit both files via **KUHUB Learn** before the deadline. Do not forget to sign the Honor Code with your name and student ID inside the notebook.

Questions Summary

Part	Topic	Key Tasks	Points
1	Modular Exponentiation Unitary	Build `make_mod_exp_gate()` as a permutation unitary; verify orbit and periodicity	25
2	Phase Estimation Circuit	Implement full QPE with controlled-$U_a^{2^k}$, correct target initialisation, and IQFT; simulate and plot	25
3	Period Extraction & Factor Pipeline	Implement `extract_period`, `validate_and_factor`, and `run_shor`; analyse per-outcome success	25
4	Generalisation to $N = 21$	Choose parameters; factor $N = 21$; sweep all coprime bases; explain RSA implications	25
	Total		**100**

Rubric (Applied to Every Part)

Each part is graded on two equal components.

- **Working Code (50 %):** The code cell(s) for the part run without errors, all assertions in the auto-check cell pass, and the outputs are numerically correct.
- **Written Explanation (50 %):** The reflection markdown cell(s) for the part contain a clear, mathematically grounded explanation of *what* the code does, *why* it is correct, and *what* the output means. A code comment such as "apply Hadamard" is not sufficient; explain the quantum-mechanical purpose of every non-trivial step.

Detailed breakdown per part:

Part	Component	Code	Explanation	Subtotal
1	Unitary construction & orbit verification	12.5	12.5	25
2	QPE circuit, simulation, & histogram	12.5	12.5	25
3	Three pipeline functions & analysis	12.5	12.5	25
4	$N = 21$ factoring, base sweep, & RSA discussion	12.5	12.5	25
	Total	**50**	**50**	**100**

Instructions

- Complete **only** the cells marked with `### YOUR CODE HERE ###` and the explanation markdown cells marked with *YOUR EXPLANATION HERE*.
- Do **not** modify any cell labelled **DO NOT MODIFY**.
- Before submission: restart the kernel and run all cells (*Kernel* → *Restart & Run All*) to ensure top-to-bottom reproducibility.
- Partial credit is awarded: a clearly written explanation of a partially working solution is valued over uncommented correct code.
- The simulation is stochastic. Auto-check probability thresholds are set conservatively so that a correct implementation will clear them reliably across random seeds.

Qiskit uses **little-endian** qubit ordering: qubit 0 is the least-significant bit. Measurement bitstrings are returned with qubit 0 at the *rightmost* position, so `int(bs, 2)` gives the correct integer value of the counting register. This convention is critical for verifying circuit outputs.

Honor Code

By submitting this assignment you affirm:

> *"I hereby certify that I have completed this assignment on my own without any help from anyone else. I have not used, accessed, received, or distributed any information from/to any other unauthorized source. The effort in this assignment belongs only to me."*

Sign with your full name and student ID in the first cell of the notebook.

Grading Notes

- This assignment contributes to the homework component of your final grade.
- Auto-check cells use `assert` statements. A failed assertion indicates a numerical error in that part; re-read the specification and debug before submitting.

- Explanation quality is assessed holistically: depth of reasoning, correct use of quantum-mechanical terminology, and connection to the relevant equations and derivations from the Week 9 notes.
- Explanation cells that merely restate the code line-by-line without conceptual content will receive **minimal** credit.